

Lab Guide 2 Module: Building and Testing Your First API Proxies

1. Introduction

Welcome to your first deep-dive lab. In this module, you'll get hands-on experience with the core function of Apigee: creating, configuring, and testing an API proxy. We will walk through the entire basic lifecycle in three parts. First, we'll create a simple "pass-through" proxy that routes traffic to a backend service. Next, we'll add a policy to transform the data. Finally, we'll use the Postman tool to test our work and see the results.

2. Learning Objectives

By the end of this lab module, you will be able to:

- Create a new reverse API proxy.
- Deploy an API proxy to an environment.
- Add a transformation policy (XML to JSON) to a proxy flow.
- Understand and manage proxy revisions.
- Test API endpoints and verify policy effects using Postman.

3. Prerequisites

- A successfully provisioned Apigee X trial instance (completed in Lab 1).
 - The Postman application installed on your computer.
 - A basic understanding of API concepts like REST, JSON, and XML.
-

Part 1: Creating a Basic API Proxy

In this part, we will create a simple reverse proxy that points to a mock backend service.

1. Navigate to the API Proxies Screen

- In the Apigee UI, use the left-hand menu to navigate to **Proxy development > API proxies**.

2. Launch the Proxy Wizard

- Click the **Create new** button.
- From the list of options, select **Reverse proxy**.

3. Enter Proxy Details

- Fill in the form with the following details:

- **Proxy Name:** music-api
- **Base Path:** /v1/music
- **Target (Existing API):** https://mocktarget.apigee.net/xml

- Click **Next**.

4. Configure Policies

- On the "Common policies" screen, leave the selection as **Passthrough (none)**. We will add policies manually later.
- Click **Next**.

5. Deploy the Proxy

- On the "Summary" screen, ensure the **eval** environment is selected for deployment.
- Click **Create and deploy**. Wait for the process to complete.

6. Confirm Deployment

- Once finished, you will see a confirmation and the proxy editor will show a green checkmark next to the eval environment, indicating a successful deployment.

Summary for Part 1: You have successfully created and deployed your first API proxy. It is now live and routing traffic from [your-domain]/v1/music to https://mocktarget.apigee.net/xml.

Part 2: Configuring Basic Policies

Now we will add logic to our proxy to transform the backend response from XML into JSON.

1. Navigate to the Develop Tab

- If you are not already there, open your **music-api** proxy from the API proxies list.
- Click on the **Develop** tab.

2. Add a Policy to the Flow

- We want to modify the response *after* it comes from the target service. In the "Navigator" pane, ensure you are on Target Endpoints > default.

- In the main "Flows" diagram, find the **Response** flow on the right side. Hover your mouse over it and click the **+ Step** button.

3. Select the "XML to JSON" Policy

- In the "Add a step" dialog, scroll through the list of policies.
- Find and select **XML to JSON**.
- Keep the default Display Name and Name. Click **Add**.

4. Save the New Revision

- You will now see the policy added to your response flow. You must save this change as a new revision.
- Click the **Save** button in the top-right corner.

5. Deploy the New Revision

- After saving, you'll see a notice that you have undeployed changes.
- Click the **Deploy revision** dropdown menu and select the **eval** environment. Confirm the deployment when prompted.

Summary for Part 2: You have successfully added a policy to your API proxy and deployed it as a new revision. The proxy now contains logic to transform data.

Part 3: Testing Endpoints with Postman

Finally, let's test our work to see the policy in action. We will use the Postman application to make requests and inspect the responses.

1. Get Your Public Proxy URL

- In the Apigee UI, navigate to your music-api proxy and click the **Debug** tab.
- Click **+ Start debug session** and ensure the eval environment is selected.
- Once the session starts, click the **Toggle URL** icon next to the hostname field.
- A dialog will appear showing two hostnames. Copy the one that ends in **.nip.io**. This is your public test URL. The other URL is for internal use and will not work from Postman.

2. Test the Backend Service Directly

- Open the Postman application.

- Create a new GET request to the direct backend URL:
`https://mocktarget.apigee.net/xml`.
- Send the request. Observe that the response body in the lower panel is in **XML** format. This is our baseline.

3. Test the Apigee Proxy

- In Postman, create another new GET request.
- In the URL field, paste your .nip.io URL. The full URL should look like this (with your specific IP): `https://[YOUR_IP_ADDRESS].nip.io/v1/music`.
- Send the request. Observe that the response body is now in **JSON** format.

4. Understand the Result

- By comparing the two tests, you can see the effect of your policy. The request to your proxy was forwarded to the backend, which returned XML. Your XML-to-JSON policy then intercepted that response, transformed it into JSON, and returned the final result to Postman.

7. Conclusion & Summary

Congratulations! In this lab module, you have successfully completed the fundamental end-to-end workflow in Apigee. You built a proxy that routes traffic to a backend, added a policy to transform the data, managed different revisions, and tested your work to verify the results.